

Fighting Numerical Hallucinations via Data-centric Compilation for Online Financial QA

Hao Chen*
FiT, Tencent
Shenzhen, China

Xing Tang*
Shenzhen Technology University
Shenzhen, China

Qirui Liu†
South China University of Technology
Guangzhou, China

Weijie Shi
The Hong Kong University of Science
and Technology
Hong Kong SAR, China

Shiwei Li
Huazhong University of Science and
Technology
Wuhan, China

Fuyuan Lyu
McGill University
Montreal, Canada

Weihong Luo
FiT, Tencent
Shenzhen, China

Xiku Du
FiT, Tencent
Shenzhen, China

Xiuqiang He‡
Shenzhen Technology University
Shenzhen, China

Abstract

Large Language Models (LLMs) have significantly advanced online data services, particularly in the domain of financial question answering (FinQA). However, such systems remain susceptible to numerical reasoning hallucinations, which critically undermine reliability in high-stakes financial applications. Although retrieval-augmented generation (RAG) has been widely adopted to ground responses in external knowledge, it introduces three persistent challenges: noise sensitivity, calculation fragility, and an auditability crisis. Existing model-centric approaches, which primarily focus on optimizing either the retriever or generator in isolation, still struggle to address these issues in an integrated manner. In this work, we pioneer a data-centric paradigm and propose a novel framework, the Data-centric Reasoning Compiler (DCRC). The framework operates through three cohesive phases: (1) adversarial data construction, which synthesizes training examples with controlled noise to teach robustness; (2) multi-stage training that cultivates a Data-centric Structuring Agent (DSA) capable of explicit evidence auditing and program synthesis; and (3) a compile-and-execute inference process, where the DSA transforms user queries and retrieved documents into verifiable, executable reasoning programs. This data-driven framework ensures faithful numerical reasoning by design. We conduct extensive experiments on established offline benchmarks and further validate our framework through deployment in a real-world online financial QA system. Notably, the framework has also been

adopted in the financial QA of YuanBao¹, one of the largest chat platforms in China.

CCS Concepts

• Information systems → Question answering; Web mining.

Keywords

Financial QA, Large Language Model, Numerical Reasoning

ACM Reference Format:

Hao Chen, Xing Tang, Qirui Liu, Weijie Shi, Shiwei Li, Fuyuan Lyu, Weihong Luo, Xiku Du, and Xiuqiang He. 2018. Fighting Numerical Hallucinations via Data-centric Compilation for Online Financial QA. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 12 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Large language models (LLMs) [1, 3] have great potential to transform online data services. A prominent and impactful application is online financial question answering (FinQA), where LLMs enhance the interpretation and response to complex user queries. However, a core challenge in deploying such systems effectively is generating answers that are both *numerically faithful* and *explainable* [18], which requires robust multi-step reasoning. The recent advent of Large Reasoning Models (LRMs) [10, 22, 27, 32], which decompose complex problems into explicit reasoning chains, provides a critical capability for addressing this challenge and building reliable online FinQA systems.

Nevertheless, the practical deployment of LRM-based FinQA systems is significantly hindered by the issue of *numerical hallucinations* [11, 34], which undermines the reliability of generated answers. To mitigate this, Retrieval-Augmented Generation (RAG) [9] has emerged as a predominant technique, grounding model responses in external, up-to-date information. However, a critical limitation persists: even with access to retrieved evidence, LRMs may still produce *unsupported reasoning steps* or present claims that *contradict* the provided content [19]. This gap between retrieval

*Both authors contributed equally to this research.

†Work done when he was an intern at FiT, Tencent.

‡Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

¹<https://yuanbao.tencent.com/chat/>

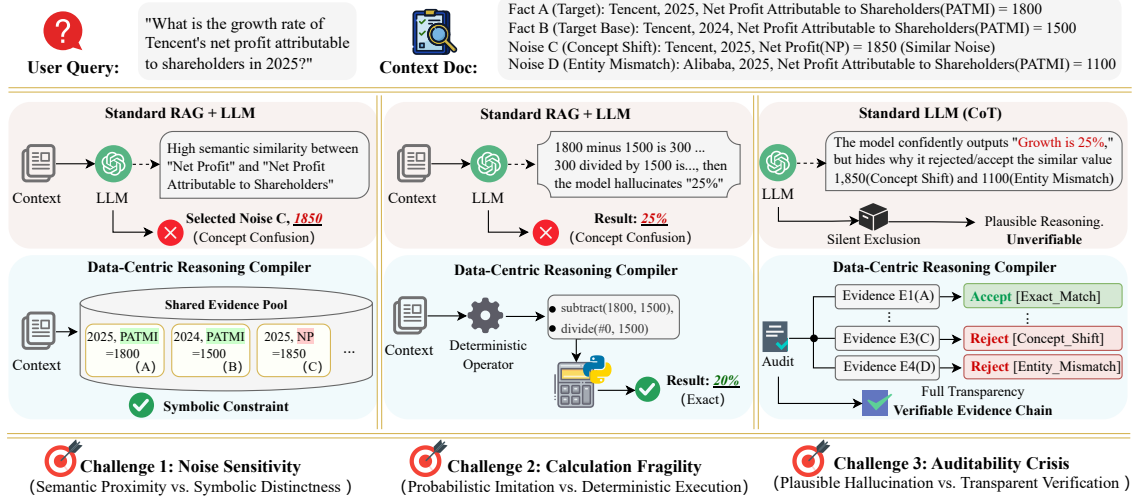


Figure 1: Illustrating the DCRC framework versus standard methods on a financial QA example. Standard approaches suffer from noise sensitivity (confusing similar facts), calculation fragility (stochastic generation), and auditability crisis (opaque reasoning). DCRC overcomes these by constructing an adversarially-augmented evidence pool, synthesizing a deterministic operator program, and generating a fully verifiable audit log.

and faithful reasoning directly impedes the development of truly robust and trustworthy FinQA systems. As shown in Figure 1, a user queries the growth rate of Tencent’s 2025 net profit attributable to shareholders. Standard RAG retrieves relevant facts but also introduces similar yet distinct noise; for example, the retrieved documents focus on net profit rather than net profit attributable to shareholders, which is semantically very similar. When relying on such noisy context, LLMs tend to incorrectly calculate the growth rate due to their inherent nature of probabilistic token prediction, which struggles to enforce deterministic arithmetic operations and faithfully align with exact numeric evidence. Even with chain-of-thought prompting [29], the model may silently omit relevant evidence without providing any verifiable reasoning to explain this exclusion.

Most existing studies on numerical reasoning adopt a retriever-generator framework [12, 23, 28], which decomposes the task into two stages: retrieval and generation. These methods focus on optimizing these two stages respectively. For example, finetuning the retriever [23], optimizing the generator [28], and prompting strategies [12]. These methods are still limited by the decoupled design. As a result, such approaches inherently fail to address the core challenges in a unified manner. Specifically, the retriever may retrieve noisy or irrelevant facts without explicit detecting potential negative samples, leading to *Noise Sensitivity*; the generator relies on probabilistic next token generation rather than deterministic computation, a problem we term *Calculation Fragility*; and the reasoning process lacks verifiable evidence chains, only leaving users with unverified answers that cause *Auditability Crisis*.

In this work, we propose a paradigm shift from a model-centric to a **data-centric** approach, and introduce the **Data-centric Reasoning Compiler (DCRC)** framework, which trains a data-centric structuring agent (DSA) to enable a trustworthy “compile-and-execute”

process. First, our framework begins by constructing a high-quality, adversarially augmented dataset through heterogeneous consensus mining and controlled noise injection, thereby teaching the DSA to directly discriminate and reject misleading facts, effectively addressing noise sensitivity. We then train the DSA with a multi-stage training. The structure-aware supervised fine-tuning (SFT) emphasizes critical auditing decisions, while reinforcement learning with verifiable multi-dimensional rewards ensures the generation of executable programs, thus eliminating calculation fragility. Finally, the trained DSA acts as a deterministic compiler at inference. DSA decomposes queries, audits evidence with explicit justifications, synthesizes structured programs, and executes them via a symbolic operator registry. The entire process yields a complete, verifiable audit log, providing full transparency and auditability. In summary, our main contributions are as follows:

- We first adopt a data-centric approach to trustworthy financial QA, which provides a practical and robust solution for online FinQA systems.
- We introduce the Data-centric Reasoning Compiler, a novel framework comprising three cohesive phases: adversarial data construction, multi-stage training, and compile-and-execute inference. DCRC transforms standard LLMs into reliable reasoning engines that effectively combat numerical hallucinations through verifiable program synthesis.
- Extensive experiments on both the offline dataset and the online scenario have been conducted to validate the superiority of our method.

2 Related work

In this section, we review two topics related to our work, including financial QA and numerical reasoning.

2.1 Financial QA

The rapid advancement of Large Language Models (LLMs) has substantially enhanced online question-answering (QA) systems, making them a core component of modern data services [14, 16, 17, 24, 33]. Financial QA, as a critical application domain, has particularly benefited from these developments, enabling users to access and analyze complex financial information in a timely manner [18]. Research in this field can be broadly categorized into two directions. The first focuses on financial numerical QA, which involves multi-step calculations and the extraction of relevant information from diverse data sources (e.g., tables, text) [25, 30, 35]. This includes zero-shot techniques that leverage LLMs’ inherent reasoning capabilities [13] to handle complex queries without task-specific training [20, 21], as well as multi-agent approaches that employ critic models to reflect on reasoning steps and answers [7]. The second direction concerns financial QA over pure text, where systems such as WeaverBird [31], a dialogue system tailored to the finance sector, leverage LLMs fine-tuned on large financial corpora to provide informed responses to user inquiries. Our work focuses on advancing numerical reasoning over retrieved tabular and textual content, aiming to provide a robust and complementary enhancement to existing financial QA systems.

2.2 Numerical Reasoning

Numerical reasoning remains a persistent challenge for large language models [2, 15, 26], and its reliable execution is central to robust financial question answering [5, 12, 23, 28]. To advance the field, several benchmarks have been developed. FinQA [5] and ConvFinQA [6] target long-form numerical reasoning over financial reports, while FinMMR [26] is a bilingual, multimodal benchmark designed to evaluate numerical reasoning capabilities in a financial context. Motivated by these benchmarks, a line of research has emerged within the prevalent retriever-generator framework. APOLLO [23] enhances the retriever via number-aware negative sampling to better discriminate key numerical facts. ENCORE [28] derives reliable reasoning processes by decomposing answer formulas. FINDER [12] employs a generative retriever for fact extraction and uses dynamic in-context example selection to improve Program-of-Thought prompting. Our proposed framework shifts to a data-centric paradigm and transforms standard LLMs into reliable reasoning engines.

3 Method

First, we provide a formal definition of our problem. Next, we elaborate on the proposed Data-centric Reasoning Compiler (DCRC) framework, which comprises three cohesive phases: adversarial data construction, multi-stage model training, and compile-and-execute inference. We will describe the design of each phase in detail.

3.1 Problem Formulation

Consider an online financial QA system powered by a Large Reasoning Model M . Given a user query q and a set of retrieved document chunks $\mathbb{D} = \{d_1, d_2, \dots, d_n\}$, the system is required to generate a numerically accurate and auditable answer a . Each document chunk d_i can be represented as a tuple $d_i = (text_i, \mathcal{V}_i)$, where $text_i$

denotes the textual content and $\mathcal{V}_i = \{v_1, v_2, \dots\}$ represents the set of candidate numerical values contained therein.

In the standard retrieval-augmented generation paradigm, the model directly generates the answer $a_g = M(q, \mathbb{D})$. However, this process is susceptible to numerical hallucinations due to noise sensitivity, calculation fragility, and auditability crisis.

Our goal is to train a Data-centric Structuring Agent (DSA) π that compiles the query and documents into a verifiable intermediate representation $\mathcal{I}$, which is subsequently executed by a deterministic executor $\mathcal{E}$ to obtain the final answer:

$$a = \mathcal{E}(\mathcal{I}), \quad \mathcal{I} = \pi(q, \mathbb{D}) \quad (1)$$

The intermediate representation $\mathcal{I}$ adheres to our designed **DSA Schema**, which mandates: (1) sub-task decomposition $\mathcal{T} = \{T_1, T_2, \dots\}$; (2) an evidence pool $\mathcal{P}$ with explicit auditing decisions; and (3) symbolic computation logic $\mathcal{L}$. This design ensures complete traceability of the reasoning process.

3.2 Adversarial Data Construction

In our DCRC framework, high-quality data is key. The data includes not only positive samples but also negative samples. To teach the model to “learn to reject” misleading evidence, we propose a systematic adversarial data synthesis pipeline comprising three steps: heterogeneous consensus mining, adversarial negative injection, and consistency filter.

3.2.1 Heterogeneous Consensus Mining. To eliminate single-model bias and hallucinations, we construct a teacher committee of K heterogeneous models $\{M_1, \dots, M_K\}$ (e.g., GPT-5.2 High², Gemini-3 Pro³, Claude-4.5 Opus⁴). For each query-document pair $(q, \mathbb{D})$, each model independently generates a structured DSA representation $\mathcal{I}_k = M_k(q, \mathbb{D})$. We then apply hard voting to determine golden labels: for each candidate value v ,

$$\text{Consensus}(v) = \begin{cases} \text{Accept}, & \text{if } \sum_{k=1}^K \mathbb{I}[\text{tag}_k(v) = \text{Accept}] \geq \lceil K/2 \rceil \\ \text{Reject}, & \text{otherwise} \end{cases} \quad (2)$$

where $\text{tag}_k(v)$ denotes the k -th model’s decision. Note that only when the majority agrees ($\geq \lceil K/2 \rceil$), and the extracted values are identical, is the evidence included in the positive set $\mathbb{B}^+$.

3.2.2 Hard Negative Injection. Instead of merely adding positive examples, we systematically inject high-difficulty negative samples that mimic real-world error patterns. This forces the model to learn not only what information to use but also what to reject, laying the foundation for verifiable reasoning. Based on positive samples, we generate high-difficulty adversarial negative samples through controlled perturbation, covering three core error patterns. Concept Shift identifies values in the document that are semantically similar to but definitionally different from the target metric; for instance, when the target is “net profit attributable to shareholders,” we construct negative samples using “operating profit” values, annotated with reason [Concept_Shift]. Temporal Mismatch maintains the metric type but substitutes values from different time periods; for example, if the query requires “2023” data,

²<https://chatgpt.com/>

³<https://gemini.google.com/>

⁴<https://claude.ai/>

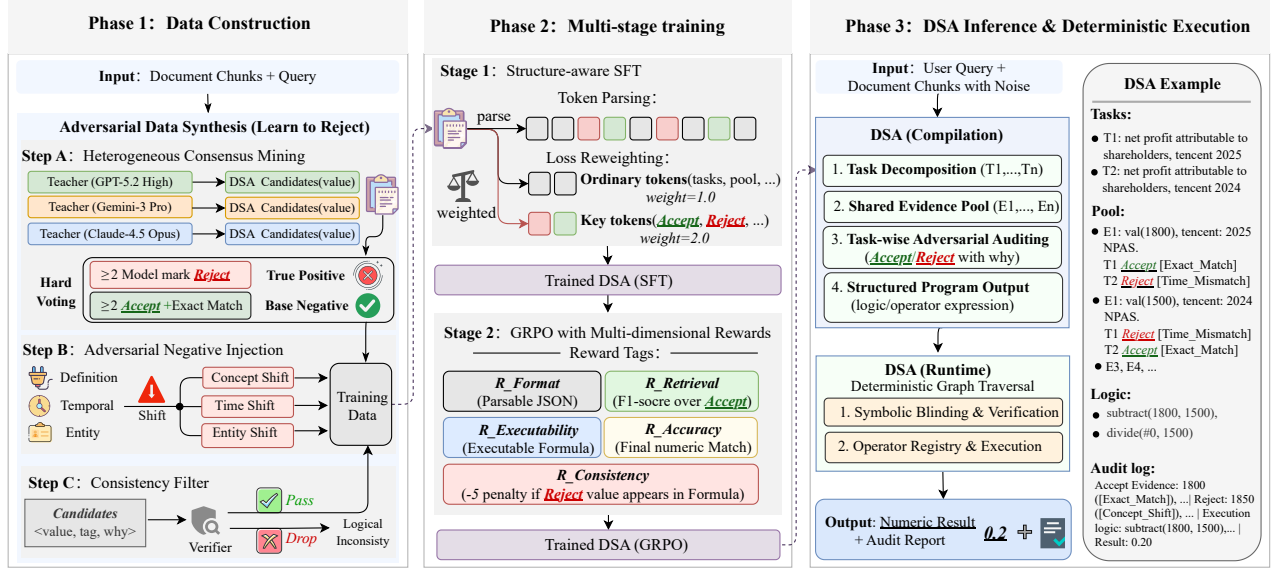


Figure 2: The overall architecture of the Data-centric Reasoning Compiler (DCRC) framework, consisting of three cohesive phases: (1) Adversarial Data Construction through heterogeneous consensus mining and hard negative injection; (2) Multi-Stage Training with structure-aware SFT and GRPO with multi-dimensional rewards; (3) Compile-and-Execute Inference where DSA generates verifiable intermediate representations executed by the deterministic CEA.

we inject values from “2022” as distractors, annotated with reason [Time_Mismatch]. Entity Confusion introduces values from different entities (e.g., subsidiary vs. group) as negative samples, annotated with reason [Entity_Mismatch].

Let the original positive sample evidence be $e^+ = (v, \text{src}, \text{ACCEPT})$, where v denotes the extracted numerical value and src represents the source text snippet from the original document where the value was found. The perturbation function ϕ_τ generates negative samples according to perturbation type $\tau \in \{\text{concept}, \text{temporal}, \text{entity}\}$:

$$e^- = \phi_\tau(e^+, \mathbb{D}) = (v', \text{src}', \text{Reject}, \text{reason}_\tau) \quad (3)$$

where v' is the perturbed distractor value, src' is the corresponding source text, and reason_τ is a predefined structured error code.

3.2.3 Consistency Filtering. To ensure the logical self-consistency of synthesized data, we introduce a verification model M_v for final cleansing. For each sample, we check the semantic consistency between its tag and reason:

$$\text{Valid}(e) = M_v(\text{tag}, \text{why}, \text{src}) \in \{0, 1\} \quad (4)$$

Only samples passing verification are included in the final training set $\mathbb{B} = \mathbb{B}^+ \cup \mathbb{B}^-$.

3.3 Multi-Stage Model Training

We adopt a two-stage training strategy to cultivate the Data-centric Structuring Agent (DSA): structure-aware supervised fine-tuning (SFT) establishes format compliance and basic discrimination capabilities, followed by reinforcement learning with multi-dimensional rewards to further optimize decision quality.

3.3.1 Structure-Aware Supervised Fine-Tuning. In the SFT stage, we fine-tune the base model using the constructed dataset $\mathbb{B}$ to master the generation of DSA-Lite Schema.

Considering that auditing decisions (Accept/Reject) are the core outputs of DSA, we apply loss weighting to the tag tokens:

$$\mathcal{L}_{\text{SFT}} = - \sum_{t=1}^T w_t \log p_\theta(y_t | y_{<t}, x) \quad (5)$$

where $p_\theta(y_t | y_{<t}, x)$ represents the probability of predicting the t -th token given the input x and the previously generated prefix $y_{<t}$ under model parameters θ . The weight w_t is defined as:

$$w_t = \begin{cases} \alpha, & \text{if } y_t \in \{\text{Accept}, \text{Reject}, \text{tag}, \text{why}\} \\ 1, & \text{otherwise} \end{cases} \quad (6)$$

We set $\alpha = 2.0$ to force the model to focus on learning classification decision boundaries.

3.3.2 Group Policy Optimization with Multi-Dimensional Rewards. In the reinforcement learning stage, we adopt the Group Relative Policy Optimization (GRPO) algorithm and design a multi-dimensional reward function to comprehensively guide model behavior.

For model output y and golden label y^* , the total reward comprises five dimensions:

(1) Format Reward $\mathcal{R}_{\text{format}}$: Checks whether the output is valid parseable JSON that complies with the schema:

$$\mathcal{R}_{\text{format}} = \begin{cases} 1, & \text{if JSON is valid and schema-compliant} \\ 0, & \text{otherwise} \end{cases} \quad (7)$$

(2) **Retrieval Reward** $\mathcal{R}_{\text{retrieval}}$: Evaluates the accuracy of evidence auditing decisions using F1 score to balance precision and recall:

$$\mathcal{R}_{\text{retrieval}} = \frac{2 \cdot P \cdot R}{P + R} \quad (8)$$

where $P = \frac{|S_{\text{pred}}^+ \cap S_{\text{gold}}^+|}{|S_{\text{pred}}^+|}$, $R = \frac{|S_{\text{pred}}^+ \cap S_{\text{gold}}^+|}{|S_{\text{gold}}^+|}$, and S^+ denotes the set of evidence marked as ACCEPT.

(3) **Consistency Penalty** $\mathcal{R}_{\text{consistency}}$: Detects logical self-consistency and severely penalizes contradictory behavior:

$$\mathcal{R}_{\text{consistency}} = \begin{cases} -\beta, & \text{if } \exists v : v \in S_{\text{pred}}^- \wedge v \in \mathcal{V}_{\text{logic}} \\ 0, & \text{otherwise} \end{cases} \quad (9)$$

where $\mathcal{V}_{\text{logic}}$ is the set of values referenced in the computation logic, and β is set to a large value (e.g., 5.0) to heavily penalize logical contradictions.

(4) **Executability Reward** $\mathcal{R}_{\text{exec}}$: Verifies whether the generated computation logic can be correctly parsed and executed by the symbolic executor:

$$\mathcal{R}_{\text{exec}} = \begin{cases} 1, & \text{if } \mathcal{E}(\mathcal{L}) \text{ executes without error} \\ 0, & \text{otherwise} \end{cases} \quad (10)$$

(5) **Accuracy Reward** $\mathcal{R}_{\text{accuracy}}$: Compares the execution result with the ground truth answer:

$$\mathcal{R}_{\text{accuracy}} = \begin{cases} 1, & \text{if } |a_{\text{pred}} - a_{\text{gold}}| < \epsilon \\ 0, & \text{otherwise} \end{cases} \quad (11)$$

Combining these above values, we get the final reward:

$$\mathcal{R}_{\text{total}} = \mathcal{R}_{\text{format}} + \mathcal{R}_{\text{retrieval}} + \mathcal{R}_{\text{exec}} + \mathcal{R}_{\text{accuracy}} + \mathcal{R}_{\text{consistency}} \quad (12)$$

For each input sample $x \in \mathbb{B}$, we sample G candidate outputs $\{y_1, y_2, \dots, y_G\}$ from the current policy π_θ . For each output y_i , we compute its reward $\mathcal{R}(y_i)$ and obtain the advantage function through group normalization: $A_i = \frac{\mathcal{R}(y_i) - \text{mean}(\{\mathcal{R}(y_j)\}_{j=1}^G)}{\text{std}(\{\mathcal{R}(y_j)\}_{j=1}^G)}$. The optimization objective is defined as:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{x \sim \mathbb{B}} \mathbb{E}_{y \sim \pi_\theta} \left[\min \left(\frac{\pi_\theta(y|x)}{\pi_{\text{old}}(y|x)} A_i, \text{clip} \left(\frac{\pi_\theta(y|x)}{\pi_{\text{old}}(y|x)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) - \beta \text{KL}(\pi_\theta \| \pi_{\text{ref}}) \right] \quad (13)$$

where π_{old} is the policy from the previous iteration, ϵ is the clipping threshold that constrains the magnitude of policy updates, β is the KL divergence penalty coefficient, and π_{ref} is the reference policy (typically the initial policy after SFT) that prevents the policy from deviating too far.

3.4 Compile-and-Execute Inference

After training, DSA serves as a deterministic compiler during inference, transforming user queries and retrieved documents into executable structured programs, which are subsequently symbolically executed to produce verifiable answers.

3.4.1 **DSA Representation.** The intermediate representation $\mathcal{I}$ generated by DSA comprises three core components. The first is task decomposition, which decomposes complex queries into atomic sub-tasks:

$$\mathcal{T} = \{T_i = (\text{target}_i, \text{constraint}_i)\}_{i=1}^m \quad (14)$$

The second is the shared evidence pool, where each piece of evidence carries multi-head decision routing, i.e., discrimination results for the same evidence under different task perspectives:

$$\mathcal{P} = \{e_j = (v_j, \text{src}_j, \{(T_i, \text{tag}_{ij}, \text{why}_{ij})\})\}_{j=1}^n \quad (15)$$

The third is symbolic computation logic, which references operators from a pre-registered operator library:

$$\mathcal{L} = \text{op}(T_1, T_2, \dots) \quad (16)$$

3.4.2 **Deterministic Symbolic Execution.** DSA executes the compiled DSA-Lite as a static computational graph through three steps. First, it performs multi-head routing aggregation by traversing the evidence pool and assigning evidence to corresponding task buckets $\mathcal{B}_i$ based on decision tags, while generating an audit log. Second, it conducts symbol binding and conflict detection through consistency verification on each task bucket:

$$\text{Bind}(T_i) = \begin{cases} v, & \text{if } |\mathcal{B}_i| = 1 \\ \text{CONFLICT}, & \text{if } |\{v : v \in \mathcal{B}_i\}| > 1 \\ \text{MISSING}, & \text{if } |\mathcal{B}_i| = 0 \end{cases} \quad (17)$$

Third, it invokes the pre-registered operator library within a safe sandbox for computation, completely eliminating code injection risks:

$$a = \text{OPERATOR_REGISTRY.execute}(\mathcal{L}, \{\text{Bind}(T_i)\}) \quad (18)$$

The entire process generates a complete audit trail, recording the acceptance reason or rejection rationale for each piece of evidence, achieving fine-grained traceability.

For example: “Accepted evidence: 59.1 ([Exact_Match]), 98.0 ([Exact_Match]) | Rejected evidence: 8.2 ([Concept_Shift]) | Computation logic: divide(59.1, 98.0) | Result: 0.60306.”

This level of explainability is difficult to achieve with traditional Chain-of-Thought methods.

4 Offline experiments

To validate the effectiveness of DCRC (Data-Centric Reasoning Compiler), we conduct comprehensive offline experiments on financial numerical reasoning benchmarks. We seek to answer the following research questions:

- **RQ1:** How does DCRC perform compared to existing SOTA methods for financial numerical reasoning?
- **RQ2:** How does the DCRC compiler paradigm compare to the traditional CoT generation paradigm?
- **RQ3:** How effective is the system’s auditability and intermediate process quality?
- **RQ4:** What are the contributions of key components in DCRC?
- **RQ5:** How do key training hyperparameters affect model performance?

4.1 Experimental settings

4.1.1 Datasets. We adopt two financial numerical reasoning benchmarks. **FinQA** [5] contains 8,281 QA pairs (train 6,251/dev 883/test 1,147), involving table-text hybrid reasoning (62.43% table-only, 23.42% text-only, 14.15% hybrid). **ConvFinQA** [6] contains 3,892 dialogues with 14,115 questions (train 3,037/dev 421/test 434), requiring multi-turn reasoning. We use the training sets from both datasets for model training and evaluate on FinQA’s dev and test sets along with ConvFinQA’s dev set (ConvFinQA test answers are private).

4.1.2 Metrics. We adopt **Execution Accuracy** as the primary metric, measuring the match rate between predicted and ground-truth results (with 1% tolerance). To evaluate DSA’s auditability, we propose three metrics: **Noise Filtering Precision** measures the proportion of true distractors among rejected evidence; **Evidence Retrieval Precision** measures the proportion of correct items among accepted evidence; **Audit Logic Accuracy** measures whether rejection reasons conform to the predefined error taxonomy.

4.1.3 Baselines. We compare with methods spanning three categories. **Prompting-based methods** include Few-shot CoT, which applies traditional chain-of-thought reasoning to elicit step-by-step solutions. **Program-based methods** include FinQANet [5], which generates domain-specific programs from financial documents; CBR-Ren [8], which leverages case-based reasoning for program retrieval and adaptation; and PoT-GPT-4 [4], which employs Program-of-Thought prompting with GPT-4 as the backbone. **Retriever-generator methods** include ArgRecog, which focuses on argument recognition for numerical reasoning; ENCORE [28], which derives reliable reasoning through answer formula decomposition; APOLLO [23], which enhances retrieval via number-aware negative sampling; and FINDER [12], which combines generative retrieval with dynamic in-context example selection. For fair comparison, FINDER uses Mistral-7B Instruct v0.2 (7B parameters) as its generator, matching the parameter scale of our DSA-7B model.

4.1.4 Implementation Details. We train DSA based on Qwen2.5 series (1.5B/3B/7B). We analyze the ACCEPT/REJECT sample distribution in the above training sets and synthesize the final training data with a 1:2 positive-negative sample ratio for data augmentation. Training consists of two stages: **Stage 1** employs structure-aware SFT with $\lambda = 2.0$ weight scaling on ACCEPT/REJECT tokens, training for 3 epochs with learning rate $5e-6$ and batch size 64; **Stage 2** applies GRPO reinforcement learning, sampling $G = 16$ outputs per input and optimizing with multi-dimensional rewards (format, retrieval F1, consistency, execution accuracy) for 3 epochs with learning rate $1e-6$.

4.2 RQ1: Comparison with SOTA Methods

To evaluate the performance gap between DCRC and existing methods, we compare DSA with various baselines, including traditional chain-of-thought methods (Few-shot CoT), program generation approaches (FinQANet), and recent SOTA methods (ENCORE, APOLLO, FINDER). The results are presented in Table 1.

Based on the results, we make the following observations. First, even our smallest 1.5B model (73.50%) surpasses ENCORE (69.40%)

Table 1: Comparison with SOTA methods. Execution Accuracy (%) is reported.

Method	FinQA-Dev	FinQA-Test	ConvFinQA-Dev
<i>Base Models (Zero-shot)</i>			
Qwen2.5-1.5B-Ins	35.10	34.22	30.15
Qwen2.5-3B-Ins	48.30	47.50	44.80
Qwen2.5-7B-Ins	61.50	60.80	59.20
<i>Existing SOTA Methods</i>			
FinQANet	61.22	61.24	68.32
ArgRecog	67.50	64.86	73.94
CBR-Ren	68.40	67.81	72.61
PoT-GPT-4	71.05	69.38	74.77
ENCORE	71.60	69.40	76.00
APOLLO	72.91	71.07	78.76
FINDER	77.13	75.32	81.95
<i>DCRC (Ours)</i>			
DSA-1.5B	73.05	73.50	71.00
DSA-3B	79.45	79.12	78.50
DSA-7B	84.71	84.66	85.67

and APOLLO (71.07%), demonstrating the efficiency advantage of the compiler paradigm through decoupling reasoning from computation. Second, our 7B model achieves 84.66% on FinQA-Test, improving upon the previous best FINDER (75.32%) by 9.34 percentage points, and reaches 85.67% on ConvFinQA with significant lead, validating the effectiveness of the DCRC architecture. Finally, DCRC achieves best performance on both FinQA and ConvFinQA, indicating good generalization ability across different financial QA scenarios.

4.3 RQ2: Compiler Paradigm vs. CoT Generation Paradigm

To verify the effectiveness of the structured compilation paradigm, we compare CoT (chain-of-thought) and DCRC (our method) across different model scales. CoT relies on implicit reasoning through free-text generation, while DCRC confines LLM uncertainty to the compilation phase (evidence filtering and structuring) while keeping the execution phase (numerical computation) deterministic. The results are presented in Table 2.

Based on the results, we draw the following key findings. First, even without any training, the data-centric compilation paradigm of DCRC demonstrates superiority over the CoT paradigm—on the 1.5B base model, DCRC achieves 35.10% while CoT only reaches 32.45%, indicating that structured output itself guides more precise information extraction. Second, after SFT+RL training, models trained with DCRC significantly outperform those trained with CoT—at 7B scale, DCRC achieves 84.66% accuracy, surpassing CoT’s 77.50% by 7.16 percentage points, validating that training models to generate verifiable structured programs is a superior paradigm. Furthermore, DCRC’s advantage persists across all model scales, demonstrating good scalability.

Table 2: Comparison of reasoning paradigms across model scales. Execution Accuracy (%) is reported.

Size	Stage	Method	FinQA-Dev	FinQA-Test	ConvFinQA-Dev
Qwen2.5-1.5B-Instruct	Base Model	Few-shot CoT	32.45	31.80	28.50
		DCRC	35.10	34.22	30.15
	SFT+RL	CoT	65.20	64.85	62.10
		DCRC	73.05	73.50	71.00
Qwen2.5-3B-Instruct	Base Model	Few-shot CoT	45.60	44.90	41.20
		DCRC	48.30	47.50	44.80
	SFT+RL	CoT	72.50	71.80	70.50
		DCRC	79.45	79.12	78.50
Qwen2.5-7B-Instruct	Base Model	Few-shot CoT	58.40	57.20	55.60
		DCRC	61.50	60.80	59.20
	SFT+RL	CoT	78.20	77.50	78.10
		DCRC	84.71	84.66	85.67

Table 3: Auditability and intermediate process quality evaluation on FinQA-Test (%).

Model	Noise Filtering Precision	Evidence Retrieval Precision	Audit Logic Accuracy
DSA-1.5B	86.4	88.2	92.5
DSA-3B	90.1	91.5	97.8
DSA-7B	97.2	96.4	99.6

4.4 RQ3: Auditability Evaluation

Auditability is a core design goal of DCRC. Unlike traditional CoT methods that only demonstrate “how to get it right”, DCRC explicitly records “why incorrect options were rejected”, providing complete audit trails for financial applications. Table 3 validates DCRC’s auditability from three dimensions.

The 7B model achieves 97.2% Noise Filtering Precision, accurately identifying and rejecting distractors—highly similar terms like “net profit attributable to parent” vs “net profit” are main error sources in traditional methods, which DCRC effectively addresses through explicit adversarial discrimination. Evidence Retrieval Precision reaches 96.4%, confirming that DSA-accepted evidence is almost always correct, preventing hallucinated data from entering computations. Audit Logic Accuracy achieves 99.6%, demonstrating that rejection reasons almost perfectly align with the predefined error taxonomy (e.g., [Time_Mismatch], [Entity_Mismatch]) rather than vague free-text explanations. These results demonstrate that DCRC not only produces correct answers but also completely explains the decision process—this “Negative Reasoning” capability is difficult for traditional CoT methods to provide.

4.5 RQ4: Ablation Study

We systematically remove key components of DCRC to evaluate their contributions. Specifically, we ablate adversarial auditing (ICCD), structured SFT, negative injection, and multi-dimensional rewards respectively. The results are shown in Table 4.

Adversarial auditing (ICCD) contributes most significantly, with removal causing the largest drop (7B: -7.1%), proving that In-Context

Table 4: Ablation study on FinQA-Test. Execution Accuracy (%) is reported. Numbers in parentheses indicate absolute drops.

Model	Full	w/o Adv. Audit	w/o Neg. Injection	w/o Multi-Dim Reward	w/o Struct. SFT
1.5B	73.50	64.10 (-9.4)	69.10 (-4.4)	70.50 (-3.0)	68.20 (-5.3)
3B	79.12	70.20 (-8.9)	75.20 (-3.9)	76.80 (-2.3)	74.50 (-4.6)
7B	84.66	77.50 (-7.1)	81.30 (-3.3)	82.90 (-1.7)	80.15 (-4.5)

Contrastive Denoising is the core mechanism—without explicit adversarial discrimination training, the model is easily confused by similar distractors. Structured SFT provides a necessary foundation for format and classification logic before reinforcement learning (7B: -4.5%). Negative injection enhances generalization (7B: -3.3%), as hard negatives force fine-grained discrimination rather than simple pattern matching. Although multi-dimensional rewards cause relatively smaller drop (7B: -1.7%), further analysis reveals their crucial role in maintaining internal logical coherence—cases of “REJECT-marked evidence appearing in final computations” increase significantly without them. These results demonstrate that DCRC components work synergistically to ensure superior performance in both accuracy and logical consistency.

4.6 RQ5: Hyperparameter Sensitivity Analysis

We analyze two key hyperparameters: the label weight coefficient λ in structured SFT and the positive-negative sample ratio in adversarial data synthesis. Results are shown in Figure 3.

Both hyperparameters exhibit inverted-U curves, with smaller models showing higher sensitivity. For λ , the 1.5B model shows sharp drops at extreme values (69.40% at $\lambda=1.0$ vs 73.50% at $\lambda=2.0$), while larger models remain relatively stable. For the sample ratio, excessive negatives (1:4) cause significant degradation especially for smaller models due to over-rejection of valid evidence. Based on comprehensive analysis, we adopt $\lambda = 2.0$ and 1:2 ratio as default configurations, achieving optimal or near-optimal performance across all scales.

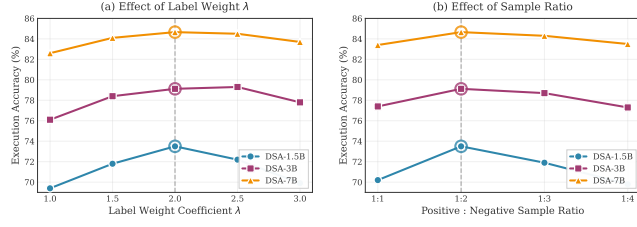


Figure 3: Hyperparameter sensitivity analysis on FinQA-Test. (a) Effect of label weight coefficient λ ; (b) Effect of positive-negative sample ratio. Circles highlight optimal configurations.

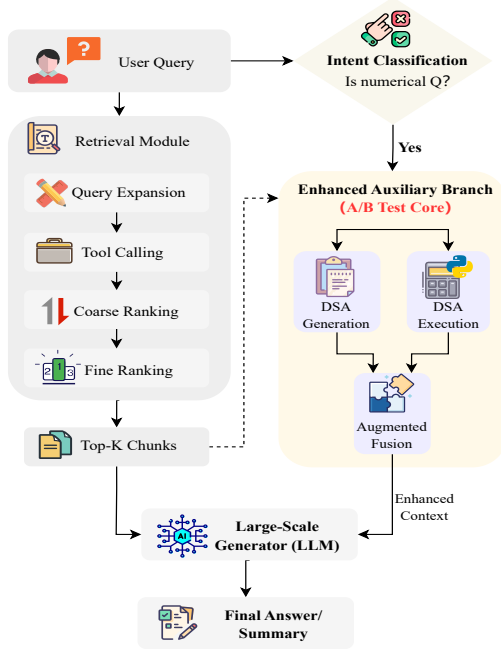


Figure 4: Architecture of the online financial QA system. Blue indicates the main pipeline; orange indicates the DSA enhancement branch (our contribution).

5 Online Experiments

To validate DCRC’s effectiveness in production, we deploy it in Tencent Yuanbao’s financial QA system for A/B testing.

5.1 Online System

As shown in Figure 4, the system consists of a main pipeline and an enhancement branch. The main pipeline retrieves top- k documents through the retrieval module (query expansion, tool calling, coarse/fine ranking) and generates answers via a large-scale LLM. The enhancement branch activates the DSA module for queries identified as financial numerical reasoning, processing retrieval results into structured evidence with audit annotations and program execution results to enhance generator input. Each DSA module

Table 5: Online A/B results (7 days, 1,000 samples per group).

Metric	Baseline	+DSA	Δ
Hallucination Rate ↓	12.44%	6.82%	-5.62%
Answer Accuracy ↑	85.26%	91.48%	+6.22%
Real-time Metric Acc. ↑	87.79%	95.69%	+7.90%
Historical Metric Acc. ↑	86.73%	93.37%	+6.64%
User Satisfaction ↑	72.00%	88.00%	+16.00%

is deployed on $2 \times H20$ GPUs (vLLM), with average output of 182 tokens and latency of 0.92s (range: 0.37s-1.24s).

5.2 Online Results

We conduct a 7-day A/B test with 9:1 traffic allocation for financial numerical reasoning queries, manually evaluating 2,000 fact claims (1,000 per group). As shown in Table 5, we report results across three dimensions: faithfulness, numerical accuracy, and user experience.

Faithfulness. DSA reduces hallucination rate from 12.44% to 6.82% (a relative reduction of 45.2%) and improves overall answer accuracy from 85.26% to 91.48%. The accuracy gain stems from both reduced hallucination and DSA’s explicit rejection of retrieval noise through structured auditing.

Numerical Accuracy. We further decompose numerical correctness into real-time metrics (e.g., current stock prices, latest quarterly earnings) and historical metrics (e.g., year-over-year growth rates, past financial indicators). DSA achieves substantial improvements on both: real-time metric accuracy increases from 87.79% to 95.69% (+7.90%), while historical metric accuracy improves from 86.73% to 93.37% (+6.64%). The gains on real-time metrics are particularly notable, as these queries are more susceptible to retrieval noise due to frequently updated data sources.

User Satisfaction. Human evaluators assess overall response quality considering correctness, completeness, and explainability. User satisfaction improves significantly from 72% to 88% (+16%), indicating that DSA’s structured audit logs and transparent reasoning enhance user trust beyond mere accuracy improvements.

Trading 0.92s additional latency for these significant quality improvements represents a reasonable trade-off in high-stakes financial scenarios where accuracy and trustworthiness are paramount.

6 Conclusion

In this paper, we present the Data-centric Reasoning Compiler (DCRC), a novel framework that addresses numerical hallucinations in online financial QA through a data-centric paradigm. DCRC comprises adversarial data construction, multi-stage agent training, and compile-and-execute inference, effectively tackling noise sensitivity, calculation fragility, and auditability crisis. We hope our work provides practitioners with practical experience in deploying trustworthy LLMs for high-stakes financial applications, and that the data-centric approach offers a new perspective beyond traditional model-centric optimizations. In the future, we plan to extend our framework to support more complex reasoning scenarios, such as multi-hop cross-document reasoning and real-time streaming financial data analysis.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Mubashara Akhtar, Abhilash Shankarampeta, Vivek Gupta, Arpit Patil, Oana Cocarascu, and Elena Simperl. 2023. Exploring the Numerical Reasoning Capabilities of Language Models: A Comprehensive Analysis on Tabular Data. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. 15391–15405.
- [3] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).
- [4] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. 2023. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. arXiv:2211.12588 [cs.CL] <https://arxiv.org/abs/2211.12588>
- [5] Zhiyu Chen, Wenhui Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan R Routledge, et al. 2021. Finqa: A dataset of numerical reasoning over financial data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 3697–3711.
- [6] Zhiyu Chen, Shiyang Li, Charese Smiley, Zhiqiang Ma, Sameena Shah, and William Yang Wang. 2022. ConvFinQA: Exploring the Chain of Numerical Reasoning in Conversational Finance Question Answering. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 6279–6292.
- [7] Sorourasadat Fatemi and Yuheng Hu. 2024. Enhancing Financial Question Answering with a Multi-Agent Reflection Framework. In *Proceedings of the 5th ACM International Conference on AI in Finance* (Brooklyn, NY, USA) (ICAIF '24). Association for Computing Machinery, 530–537.
- [8] Boda Feng, Hui Gao, Peng Zhang, and Jing Zhang. 2024. CBR-Ren: A Case-Based Reasoning Driven Retriever-Generator Model for Hybrid Long-Form Numerical Reasoning. Springer-Verlag, Berlin, Heidelberg, 111–126. doi:10.1007/978-3-031-63646-2_8
- [9] Yunfan Gao, Yun Xiong, Xinyu Gao, Changxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* 2, 1 (2023).
- [10] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature* 645, 8081 (2025), 633–638.
- [11] Haoqiang Kang and Xiao-Yang Liu. 2023. Deficiency of large language models in finance: An empirical examination of hallucination. *arXiv preprint arXiv:2311.15548* (2023).
- [12] Subhendu Khatuya, Shashwat Naidu, Pawan Goyal, and Niloy Ganguly. 2025. Program of Thoughts for Financial Reasoning: Leveraging Dynamic In-Context Examples and Generative Retrieval. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Suzhou, China, 31006–31018.
- [13] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. *Advances in neural information processing systems* 35 (2022), 22199–22213.
- [14] Hanyu Lai, Xiao Liu, Hao Yu, Yifan Xu, Iat Long long, Shuntian Yao, Aohan Zeng, Zhengxiao Du, Yuxiao Dong, and Jie Tang. 2025. WebGLM: Towards an Efficient and Reliable Web-Enhanced Question-Answering System. *ACM Trans. Inf. Syst.* 43, 5, Article 122 (July 2025), 43 pages.
- [15] Haoyang Li, Xuejia Chen, Zhanchao Xu, Darian Li, Nicole Hu, Fei Teng, Yiming Li, Luyu Qiu, Chen Jason Zhang, Li Qing, and Lei Chen. 2025. Exposing Numeracy Gaps: A Benchmark to Evaluate Fundamental Numerical Abilities in Large Language Models. In *Findings of the Association for Computational Linguistics: ACL 2025*. 20004–20026.
- [16] Xin Lin, Zhenya Huang, Zhiqiang Zhang, Jun Zhou, and Enhong Chen. 2025. Explore What LLM Does Not Know in Complex Question Answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 24585–24594.
- [17] Langming Liu, Shilei Liu, Yujin Yuan, Yizhen Zhang, Bencheng Yan, Zhiyuan Zeng, Zihao Wang, Jiaqi Liu, Di Wang, Wenbo Su, et al. 2025. UQABench: Evaluating User Embedding for Prompting LLMs in Personalized Question Answering. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 5652–5661.
- [18] Yuqi Nie, Yaxuan Kong, Xiaowen Dong, John M Mulvey, H Vincent Poor, Qingsong Wen, and Stefan Zohren. 2024. A survey of large language models for financial applications: Progress, prospects and challenges. *arXiv preprint arXiv:2406.11903* (2024).
- [19] Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, KaShun Shum, Randy Zhong, Juntong Song, and Tong Zhang. 2024. RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 10862–10878.
- [20] Karmvir Singh Phogat, Chetan Harsha, Sridhar Dasaratha, Shashishekar Ramakrishna, and Sai Akhil Puranam. 2023. Zero-Shot Question Answering over Financial Documents using Large Language Models. *CoRR abs/2311.14722* (2023). <https://doi.org/10.48550/arXiv.2311.14722>
- [21] Karmvir Singh Phogat, Sai Akhil Puranam, Sridhar Dasaratha, Chetan Harsha, and Shashishekar Ramakrishna. 2024. Fine-tuning Smaller Language Models for Question Answering over Financial Documents. In *Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, Florida, USA, November 12–16, 2024*. Association for Computational Linguistics, 10528–10548.
- [22] Aske Plaat, Annie Wong, Suzan Verberne, Joost Broekens, Niki Van Stein, and Thomas Bäck. 2025. Multi-Step Reasoning with Large Language Models, a Survey. *ACM Comput. Surv.* 58, 6, Article 160 (Dec. 2025), 35 pages.
- [23] Jiashuo Sun, Hang Zhang, Chen Lin, Xiangdong Su, Yeyun Gong, and Jian Guo. 2024. APOLLO: An Optimized Training Approach for Long-form Numerical Reasoning. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, Nicoletta Calzolari, Min-Yen Kan, Veronique Hoste, Alessandro Lenci, Sakriani Sakti, and Nianwen Xue (Eds.). 1370–1382.
- [24] Xing Tang, Hao Chen, Shiwei Li, Fuyuan Lyu, Weijie Shi, Lingjie Li, Dugang Liu, Weihong Luo, Xiku Du, and Xiuqiang He. 2026. Data-Driven Function Calling Improvements in Large Language Model for Online Financial QA.
- [25] Zichen Tang, Haihong E, Ziyang Ma, Haoyang He, Jiacheng Liu, Zhongjun Yang, Zihua Rong, Rongjin Li, Kun Ji, Qing Huang, Xinyang Hu, Yang Liu, and Qianhe Zheng. 2025. FinanceReasoning: Benchmarking Financial Numerical Reasoning More Credible, Comprehensive and Challenging. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 15721–15749.
- [26] Zichen Tang, Jiacheng Liu, Zhongjun Yang, Rongjin Li, Zihua Rong, Haoyang He, Zhuodi Hao, Xinyang Hu, Kun Ji, Ziyang Ma, et al. 2025. Finmmr: make financial numerical reasoning more multimodal, comprehensive, and challenging. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 3245–3257.
- [27] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. 2025. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599* (2025).
- [28] Dingzirui Wang, Longxu Dou, Xuanliang Zhang, Qingfu Zhu, and Wanxiang Che. 2024. Enhancing Numerical Reasoning with the Guidance of Reliable Reasoning Processes. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 10812–10828.
- [29] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*. Article 1800, 14 pages.
- [30] Qianqian Xie, Weiguang Han, Zhengyu Chen, Ruoyu Xiang, Xiao Zhang, Yueru He, Mengxi Xiao, Dong Li, Yongfu Dai, Duanyu Feng, et al. 2024. Finben: A holistic financial benchmark for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 95716–95743.
- [31] Siqiao Xue, Fan Zhou, Yi Xu, Ming Jin, Qingsong Wen, Hongyan Hao, Qingyang Dai, Caigao Jiang, Hongyu Zhao, Shuo Xie, et al. 2023. Weaverbird: Empowering financial decision-making with large language model, knowledge base, and search engine. *arXiv preprint arXiv:2308.05361* (2023).
- [32] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [33] Murong Yue. 2025. A survey of large language model agents for question answering. *arXiv preprint arXiv:2503.19213* (2025).
- [34] Mengao Zhang, Jiayu Fu, Tanya Warrior, Yuwen Wang, Tianhui Tan, and Ke-wei Huang. 2025. FAITH: A Framework for Assessing Intrinsic Tabular Hallucinations in Finance. 159–167.
- [35] Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. 2021. TAT-QA: A Question Answering Benchmark on a Hybrid of Tabular and Textual Content in Finance. In *ACL Association for Computational Linguistics*.

A Prompt used in Method

Prompt: DSA (Data-centric Structuring Agent)

Role:

You are a professional financial numerical reasoning analyst. Your task is to analyze financial documents and tables to answer users' financial numerical calculation questions.

Task Description:

Given a financial document (containing text and tables) and a question, you need to:

1. **Understand the question:** Identify the target metric and constraints required for calculation
2. **Extract evidence:** Extract all potentially relevant/confusable numerical evidence from the document
3. **Discriminate and filter:** For each piece of evidence, decide whether to accept it with structured reasons
4. **Construct calculation:** Determine computation logic and output the final answer

Output Format:

Please strictly follow the JSON format below:

```
{
  "tasks": {
    "T1": { "target": "target metric name", "constraint": "time/condition" },
    "T2": { "target": "target metric name", "constraint": "time/condition" }
  },
  "pool": [
    {
      "id": "E1",
      "val": "value (keep original format)",
      "src": "source text snippet",
      "decisions": {
        "T1": { "tag": "ACCEPT", "why": "[Exact_Match]" },
        "T2": { "tag": "REJECT", "why": "[Time_Mismatch]" }
      }
    }
  ],
  "logic": "calculation expression"
}
```

Decision Tag Descriptions:

ACCEPT Reason Codes:

- [Exact_Match] - Exactly matches target metric and all constraints
- [Derived_Match] - Can be derived through simple transformation

REJECT Reason Codes:

- [Time_Mismatch] - Time/year does not match
- [Entity_Mismatch] - Entity/company name does not match
- [Concept_Shift] - Different metric concept (e.g., operating profit vs. net profit)
- [Unit_Error] - Unit mismatch
- [Irrelevant] - Value irrelevant to the question

Supported Calculation Operations:

- add(a, b) - Addition
- subtract(a, b) - Subtraction (a - b)
- multiply(a, b) - Multiplication
- divide(a, b) - Division (a / b)
- exp(a, b) - Exponentiation (a ^ b)
- greater(a, b) - Comparison (returns "yes" or "no")

Use #N to reference the result of step N (starting from 0), e.g.:

- subtract(100, 80), divide(#0, 80) means first compute 100-80=20, then 20/80=0.25

B Illustrative Example: DSA Evidence Auditing

Example: DSA Output for Financial Numerical Reasoning

Question:

What was the difference in percentage return for PMI common stock compared to the S&P 500 index for the five years ended 2018?

Gold Answer: -0.538

DSA Output:

```
{
  "tasks": {
    "T1": { "target": "pmi", "constraint": "december 31 2018" },
    "T2": { "target": "s&p 500 index", "constraint": "december 31 2018" }
  },
  "pool": [
    { "id": "E1", "val": "96.50",
      "src": "december 31 2018 | $96.50 | $127.70 | $150.30",
      "decisions": {
        "T1": { "tag": "ACCEPT", "why": "[Exact_Match]" },
        "T2": { "tag": "REJECT", "why": "[Concept_Shift]" } } },
    { "id": "E2", "val": "150.30",
      "src": "december 31 2018 | $96.50 | $127.70 | $150.30",
      "decisions": {
        "T1": { "tag": "REJECT", "why": "[Concept_Shift]" },
        "T2": { "tag": "ACCEPT", "why": "[Exact_Match]" } } },
    { "id": "E3", "val": "100.00",
      "src": "december 31 2013 | $100.00 | $100.00 | $100.00",
      "decisions": { "ALL": { "tag": "REJECT", "why": "[Time_Mismatch]" } } },
    { "id": "E4", "val": "127.70",
      "src": "december 31 2018 | $96.50 | $127.70 | $150.30",
      "decisions": { "ALL": { "tag": "REJECT", "why": "[Concept_Shift]" } } },
    { "id": "E5", "val": "144.50",
      "src": "december 31 2017 | $144.50 | $140.50 | $157.20",
      "decisions": { "ALL": { "tag": "REJECT", "why": "[Time_Mismatch]" } } }
  ],
  "logic": "subtract(96.50, const_100), divide(#0, const_100),
    subtract(150.30, const_100), divide(#2, const_100),
    subtract(#1, #3)"}
```

Audit Log:

Accepted evidence: 96.50 ([Exact_Match]), 150.30 ([Exact_Match]) | Rejected evidence: 96.50 ([Concept_Shift]), 150.30 ([Concept_Shift]), 100.00 ([Time_Mismatch]), 127.70 ([Concept_Shift]), 144.50 ([Time_Mismatch]) | Computation logic: subtract(96.50, const_100), divide(#0, const_100), subtract(150.30, const_100), divide(#2, const_100), subtract(#1, #3) | Result: -0.538

C Illustrative Example: DSA Program Execution

Operator Registry and Deterministic Execution Process

Registered Operator Library:

```
OPERATOR_REGISTRY = {
    "add": lambda a, b: a + b,
    "subtract": lambda a, b: a - b,
    "multiply": lambda a, b: a * b,
    "divide": lambda a, b: a / b,
    "exp": lambda a, b: a ** b,
    "greater": lambda a, b: "yes" if a > b else "no",
    "table_max": lambda row: max(row),
    "table_min": lambda row: min(row),
    "table_sum": lambda row: sum(row),
    "table_avg": lambda row: sum(row) / len(row)
}
```

Execution Example:

Input Program: subtract(96.50, const_100), divide(#0, const_100), ...

Step-by-Step Execution:

```
# Step 0: subtract(96.50, const_100)
>>> arg1, arg2 = 96.50, 100.0
>>> res[0] = OPERATOR_REGISTRY["subtract"](arg1, arg2)
>>> res[0] = -3.50

# Step 1: divide(#0, const_100)
>>> arg1, arg2 = res[0], 100.0 # #0 references Step 0
>>> res[1] = OPERATOR_REGISTRY["divide"](arg1, arg2)
>>> res[1] = -0.035

# Step 2: subtract(150.30, const_100)
>>> arg1, arg2 = 150.30, 100.0
>>> res[2] = OPERATOR_REGISTRY["subtract"](arg1, arg2)
>>> res[2] = 50.30

# Step 3: divide(#2, const_100)
>>> arg1, arg2 = res[2], 100.0
>>> res[3] = OPERATOR_REGISTRY["divide"](arg1, arg2)
>>> res[3] = 0.503

# Step 4: subtract(#1, #3)
>>> arg1, arg2 = res[1], res[3]
>>> res[4] = OPERATOR_REGISTRY["subtract"](arg1, arg2)
>>> res[4] = -0.538

# Final Result: round(-0.538, 5) = -0.538
```

Key Design Principles:

1. **Sandbox Execution:** All operations invoke pre-registered functions only, eliminating arbitrary code injection risks.
2. **Reference Resolution:** #N tokens are resolved to previous step results, enabling multi-step computation.
3. **Deterministic Output:** Results are rounded to 5 decimal places for reproducibility.